IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

A Constituent Unit of Yenepoya (Deemedtobe University)

Balmatta, Mangalore

IBM PROJECTREPORT

ON

"Design and Development of Natural Language Interface for EdTech Infrastructure"

SUBMITTED BY

MOHAMMED FAYIS

III BCA (Computer Applications)

Reg No: 23BCAICD059

UNDER THE GUIDANCE OF

MS Deepthi

Department of Computer Science

IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF

BACHELOR OFCOMPUTER APPLICATIONS

MAY 2026

CERTIFICATE

This is to certify that the project entitled "Design and Development of Natural Language Interface for EdTech Infrastructure" submitted by Mohammed Fayis, Reg No: 23BCAICD059, III BCA, Department of Computer Science, Yenepoya Institute of Arts, Science, Commerce and Management, Mangalore, is a bonafide record of the work done by the student in partial fulfillment of the requirement for the award of the Degree of Bachelor of Computer Applications under Yenepoya (Deemed to be University), during the academic year 2025-2026.

The project has been carried out under our guidance and supervision. The results embodied in this report have not been submitted to any other University or Institution for the award of any degree or diploma.

_____
Internal Guide
Department of Computer Science

_____
Head of Department
Department of Computer Science

_____
Principal
Yenepoya InstituteofArts,Science,Commerceand Management

DECLARATION

I, Mohammed Fayis, Reg No: 23BCAICD059, student of III BCA, Department of Computer Science, Yenepoya Institute of Arts, Science, Commerce and Management, Mangalore, hereby declare that the project entitled "Design and Development of Natural Language Interface for EdTech Infrastructure" submitted to Yenepoya (Deemed to be University) in partial fulfillment of the requirement for the award of the Degree of Bachelor of Computer Applications is my original work.

I further declare that this project or any part thereof has not been submitted for the award of any degree or diploma of this University or any other University or Institution. The information and data given in the report is authentic to the best of my knowledge.

The project report is prepared in compliance with the guidelines issued by the Department of Computer Science, Yenepoya Institute of Arts, Science, Commerce and Management.

Place: Mangalore
Date: May 2026

_____
Mohammed Fayis
Reg No: 23BCAICD

ACKNOWLEDGEMENT

I express my deep sense of gratitude and sincere thanks to all those who have directly or indirectly helped me throughout the successful completion of this project.

I am grateful to the Principal of Yenepoya Institute of Arts, Science, Commerce and Management for providing the necessary infrastructure and facilities to undertake this IBM project.

I express my heartfelt thanks to the Vice Principal for the moral support and encouragement extended throughout the project.

I am deeply indebted to the Head of the Department of Computer Science for the constant guidance, motivation and support provided during the entire course of the project.

My sincere thanks go to my Internal Guide for their invaluable guidance, timely suggestions, constructive criticism and continuous encouragement throughout the project. Their insights into natural language processing, software design and educational technology helped shape the final outcome of this work.

I extend my gratitude to all the faculty members of the Department of Computer Science for their support and advice during various stages of this project.

I am also grateful to IBM for providing the IBM mentorship opportunity and access to mentors and resources that significantly enriched the technical depth of this project.

Last but not least, I thank my family and friends for their patience, understanding and unwavering moral support throughout this endeavor.

Fayis

TABLE OF CONTENTS

Section	Page No.
Executive Summary	6
1. Background	7
2. Aim and Objectives	9
3. Technologies Used	1
4. Hardware Architecture	1
5. Software Architecture	1
6. System Requirements	4
7. Design and Architecture	1
8. Implementation	6
9. Testing	1
10. Graphical User Interface (GUI) Layout	9
11. Evaluation	2
12. Snapshots of the Project	2
13. Conclusions	2
14. Further Development or Research	7
15. References	3
16. Appendix	5
	3
	9
	4
	2
	4
	4
	4
	6
	4
	7
	4
	9
	5
	0

EXECUTIVE SUMMARY

This report documents the design and development of a Natural Language Interface (NLI) for EdTech Infrastructure, built as part of an IBM project in association with IBM. The project, titled EdTech Chatbot, is a web-based academic assistant that allows students to interact with their educational data using natural language queries rather than navigating complex portals.

Background

Educational Technology (EdTech) platforms have significantly transformed the delivery of education worldwide. However, a persistent challenge remains: students find it difficult to quickly access their academic information such as grades, attendance, fees, and assignments from portal interfaces that require navigating multiple menus. A conversational chatbot powered by Natural Language Processing (NLP) addresses this usability gap.

Aim

The primary aim of this project is to design and develop a hybrid Natural Language Interface that allows college students to query their academic information in plain English, eliminating the need for navigating traditional academic portals. The interface combines rule-based intent matching for speed and accuracy with a Large Language Model (LLM) fallback for open-ended academic queries.

Technologies

The system is implemented using Python 3.11 as the core programming language, Streamlit as the web UI framework, Groq API for LLM inference with the Llama 3.3 70B model, and python-dotenv for environment configuration. The architecture employs a hybrid AI model combining keyword-based intent classification with a large language model fallback.

Hardware Architecture

The system is designed for deployment on standard commodity hardware. The client accesses the system through any modern web browser. The server component requires a minimum of 4 GB RAM, a dual-core processor, and a stable internet connection for LLM API calls.

Software Architecture

The software architecture follows a modular three-layer design: the Presentation Layer (Streamlit UI), the Business Logic Layer (chatbot engine and LLM integration), and the Data Layer (student profile and academic records). The system processes user queries through a pipeline of intent classification, data retrieval, and response formatting.

Key Outcomes

The completed system successfully delivers a fully functional, aesthetically refined chatbot interface that handles eight core academic intents with 100% rule-based accuracy and falls back gracefully to a state-of-the-art LLM for open-ended queries. The system was tested across functional, usability, and performance dimensions with satisfactory results.

1. BACKGROUND

1.1 Introduction to EdTech and Conversational Interfaces

Theeducation sector has witnessedaprofound digital transformation over the past decade. Educational Technology, commonly referred to as EdTech, encompasses a broad range of digital tools, platforms and services that support the delivery, management and enhancement of education. From Learning Management Systems (LMS) to online examination portals, EdTech platforms have become integral to the modern academic experience.

Despite the widespread adoption of these platforms, student engagement and usability remain critical challenges. Traditional academic portals are often designed with information density as a priority, requiring students to navigate through multiple hierarchical menus to access basic information such as their course enrollment, attendance records, grade sheets and fee statements. This fragmented user experience leads to frustration and inefficiency, particularly among students who may already be under academic pressure.

Conversational interfaces, popularly known as chatbots, offer a compelling solution to this usability challenge. By enabling users to interact with digital systems using natural language, chatbots eliminate the learning curve associated with complex portal navigation. The user simply types or speaks a query in their natural language, and the system responds with the relevant information in a concise and readable format.

1.2 The Role of Natural Language Processing in Education

NaturalLanguage Processing(NLP) is a subfield of Artificial Intelligencethat focuses on enabling computers to understand, interpret and generate human language. NLP techniques form the computational backbone of conversational interfaces and have matured significantly with the advent of transformer-based language models.

In the context of education, NLP-powered chatbots serve multiple functions: they can answer frequently asked questions, provide personalized feedback, assist in study

planning, explain academic concepts and retrieve student-specific data. The application of NLP in EdTech represents a significant step toward personalized, accessible and responsive educational support.

This project leverages NLP at two levels. At the first level, a lightweight keyword-based intent classification mechanism provides fast and accurate responses to common, well-defined queries. At the second level, a full Large Language Model (LLM) - specifically Meta's Llama 3.3 70B served through Groq's inference API - handles open-ended, nuanced or context-dependent queries that cannot be addressed by simple pattern matching.

1.3 Problem Statement

College students frequently require quick access to academic information including enrolled courses, semester grades, attendance percentages, fee payment status and assignment deadlines. Existing academic portals at most institutions require students to:

- Log into the portal with credentials
- Navigate to a specific module (e.g., academic records, finance, assignments)
- Apply filters or search for relevant information
- Interpret raw data from tables or charts

This multi-step process is inefficient for routine information retrieval. Students who need a quick answer - for example, whether their attendance is adequate or whether a fee payment deadline has passed - must invest disproportionate time and effort.

Furthermore, when students have academic questions that fall outside the scope of simple data retrieval - such as asking for help understanding a concept from their curriculum, planning a study schedule or seeking guidance on pending assignments - existing portals offer no assistance whatsoever.

1.4 Motivation for the Project

The motivation for this project arises from the confluence of three trends. First, the increasing prevalence of conversational AI in consumer applications has raised user expectations: students accustomed to interacting with assistants like Siri, Google

Assistant or ChatGPT naturally expect similar interfaces from their academic platforms. Second, the availability of high-quality, free-tier LLM APIs (particularly through Groq's platform) makes it technically feasible to build sophisticated conversational interfaces at minimal cost. Third, the growth of Python web frameworks like Streamlit has dramatically lowered the barrier to deploying polished, production- ready web applications.

This project demonstrates that a capable, attractive and intelligent academic chatbot can be built and deployed by a small development team using readily available open-source tools and free-tier APIs.

2. AIM AND OBJECTIVES

2.1 Primary Aim

Theprimaryaim of this project is to design, develop and deploy a Natural Language Interface (NLI) for an EdTech infrastructure that enables college students to access their academic information and receive educational assistance through a conversational web interface. The system should be intuitive, responsive and accurate, providing a superior user experience compared to traditional academic portal navigation.

2.2 Specific Objectives

Thefollowing specific objectives were defined at the outset of the project:

1. Design a hybrid intent classification system that combines rule-based keyword matching for high-frequency, well-defined queries with an LLM fallback for open-ended and complex queries.
2. Develop a comprehensive student data model encompassing course enrollment, grade records, attendance data, fee information and assignment tracking.
3. Implement a Streamlit-based web interface that provides a modern, accessible and responsive chat experience with a dark sidebar, conversation history and quick-access chips.
4. Integrate the Groq API to leverage the Llama 3.3 70B large language model for intelligent, context-aware responses to open-ended academic queries.
5. Inject student profile data as system context into LLM prompts to ensure that AI-generated responses remain grounded in the student's actual academic records.
6. Support dual deployment modes: a graphical web interface via Streamlit and a command-line interface (CLI) for terminal-based access.
7. Design the system architecture with modularity and separation of concerns so that each component (UI, chatbot logic, LLM integration) can be independently tested and maintained.
8. Ensure graceful degradation: the system should remain fully functional for rule-based queries even when the LLM API key is not configured.

2.3 Scope of the Project

Theproject scope encompasses the design, implementation and testing of a single-user academic chatbot tailored for a Computer Science and Engineering student in their fourth semester. While the current implementation uses static, hardcoded student data, the system architecture is designed to accommodate dynamic database integration in future iterations.

The system handles the following query categories within its defined scope:

- Course enrollment information
- Semester grade records
- Subject-wise attendance summary and warnings
- Fee payment status and balance
- Assignment list, due dates and submission status
- Quick status dashboard combining all data
- General academic and computer science study questions via LLM

The system is explicitly designed for deployment as a local web application, accessible via a browser at localhost. Cloud deployment, multi-user authentication and real-time database synchronization are identified as areas for future development.

2.4 Expected Outcomes

Uponsuccessful completionof the project, the following outcomes were expected:

- A fully functional, aesthetically refined chatbot web application deployable with a single command.
- A modular Python codebase that cleanly separates presentation, business logic and data concerns.
- Accurate handling of all eight defined intents with appropriate Markdown-formatted responses.
- Intelligent, context-aware responses to open-ended academic queries powered by Llama 3.3 70B.
- A documented, well-commented codebase suitable for academic submission and future extension.

3. TECHNOLOGIES USED

3.1 Programming Language: Python 3.11

Pythonis the primary programming language used in this project. Chosen for its rich ecosystem of libraries for web development, natural language processing and API integration, Python 3.11 provides the foundation for all components of the system. Its readable syntax and dynamic typing allow for rapid development and prototyping, while its extensive standard library and third-party packages support every aspect of the implementation.

Python 3.11 introduced several performance improvements, including a significantly faster interpreter, and new exception group features that improve error handling. These improvements contribute to the system's responsiveness and reliability.

3.2 Streamlit Framework

Streamlit is an open-source Python framework that transforms Python scripts into shareable web applications with minimal effort. Unlike traditional web frameworks such as Flask or Django, which require separate frontend code in HTML, CSS and JavaScript, Streamlit allows developers to define the entire user interface within Python using a declarative API.

In this project, Streamlit provides:

- The chat message rendering interface (st.chat_message, st.chat_input)
- Sidebar layout with model selection and student profile display
- Session state management for conversation history persistence
- Real-time UI updates through st.rerun() for chip-triggered queries
- Spinner indicators for LLM response loading

Custom CSS injection through st.markdown(unsafe_allow_html=True) enables the dark sidebar, gradient toolbar, pill status badges and side-tab message bubbles that give the application its distinctive visual identity. The use of CSS variables for theming ensures visual consistency across all components.

3.3 Groq API and Llama 3.3 70B

Groq is a cloud inference platform that provides extremely fast inference for open-source large language models. The Groq API exposes these models through an OpenAI-compatible REST interface, making integration straightforward. In this project, Groq is used to access Meta's Llama 3.3 70B Versatile model.

Llama 3.3 is Meta AI's third-generation large language model series. The 70 billion parameter variant offers a strong balance between response quality and inference speed. Key characteristics relevant to this project include:

- Strong instruction following: the model accurately follows system prompt guidelines regarding tone, format and content scope.
- Markdown generation: the model reliably produces well-formatted Markdown responses including tables, bullet lists and code blocks.
- Context retention: over a conversation, the model maintains awareness of previously exchanged messages when the full history is provided.
- Academic knowledge: the model has extensive knowledge of computer science curricula, programming concepts and academic best practices.

The integration is implemented through the official groq Python library. The client is instantiated with the API key from the environment, and completions are requested with a temperature setting of 0.5 to balance creativity and factual accuracy. Responses are capped at 600 tokens to maintain conciseness.

3.4 python-dotenv

The python-dotenv library provides a mechanism for loading environment variables from a .env file into the application's runtime environment. This is used in the project to manage the GROQ_API_KEY securely, keeping the API key out of the source code and preventing accidental exposure in version control systems.

3.5 Google Fonts: Inter and Fraunces

Thevisual designof the application usestwo custom typefaces loaded from Google Fonts via CDN. Inter is a sans-serif typeface optimized for screen legibility, used for

body text, labels, sidebar elements and the chat input. Fraunces is a display serif typeface used for prominent headings, providing a premium editorial feel. Together, they create a visual hierarchy that is both functional and distinctive.

3.6 Technology Stack Summary

Component	Technology	Version/Detail
Core Language	Python	3.11
Web Framework	Streamlit	>= 1.32.0
LLM Provider	Groq + Llama 3.3	llama-3.3-70b-versatile
LLM Client	groq Python SDK	>= 0.11.0
Environment	python-dotenv	>= 1.0.0
UI Fonts	Google Fonts	Inter, Fraunces
Styling	Custom CSS via HTML	Injected via Streamlit

4. HARDWARE ARCHITECTURE

4.1 Overview

The EdTech Chatbot system operates on a client-server model. The server component runs the Streamlit application and handles all business logic and LLM API communication. The client component is any device with a modern web browser, which accesses the application over the local network or internet. The use of Streamlit abstracts the web server infrastructure, automatically creating a WebSocket-based connection between the browser client and the Python backend.

4.2 Development and Deployment Hardware

During development,thesystemwasbuiltandtestedon a standard laptop computer with the following specifications:

Hardware Component	Specification
Processor	Intel Core i5 (10th Gen) or equivalent
RAM	8 GB DDR4
Storage	256 GB SSD
Network	Wi-Fi 802.11ac / Ethernet 100 Mbps
Operating System	Windows 10 / Ubuntu 22.04 LTS
Browser	Google Chrome 120+ / Firefox 121+

4.3 Minimum Hardware Requirements

Thefollowingare theminimumhardwarerequirements for deploying and running the EdTech Chatbot application:

- Processor: Dual-core CPU at 1.8 GHz or faster
- RAM: 4 GB (8 GB recommended for smooth performance)
- Storage: 500 MB free disk space for the application and dependencies
- Network: Stable internet connection (minimum 5 Mbps) for LLM API calls

- Browser: Any modern browser supporting WebSocket connections (Chrome, Firefox, Safari, Edge)

4.4 Network Architecture

The system's network architecture involves three primary communication pathways:

9. Client to Streamlit Server: The user's browser communicates with the Streamlit server running on the local machine (or a cloud host) via HTTP/WebSocket on port 8501. Streamlit's WebSocket protocol enables real-time bidirectional updates, allowing the UI to refresh dynamically as the chatbot generates responses.
10. Streamlit Server to Groq API: When the chatbot's rule-based engine cannot classify a user query, the server makes a secure HTTPS POST request to the Groq API endpoint at api.groq.com. The request includes the student context, conversation history and user message. The response is returned as a JSON payload containing the LLM-generated text.
11. Google Fonts CDN: The application loads the Inter and Fraunces typefaces from Google Fonts CDN over HTTPS on initial page load. This requires internet access but does not affect the chatbot's core functionality.

4.5 Cloud Deployment Considerations

While the current implementation is designed for local deployment, the architecture is compatible with cloud hosting platforms. Streamlit Community Cloud offers free hosting for public Streamlit applications, requiring only a GitHub repository and a Streamlit Cloud account. For institutional deployment, platforms such as Heroku, AWS EC2, Google Cloud Run or Azure App Service can host the application with appropriate scaling configurations. The LLM component's API-based design means there is no need to host the model locally, significantly reducing infrastructure costs.

5. SOFTWARE ARCHITECTURE

5.1 Architectural Overview

TheEdTech Chatbotemploys a modular, layered software architecture that cleanly separates concerns across three primary components. This separation allows each component to be independently tested, modified and extended without affecting the others. The architecture follows the principles of high cohesion within modules and low coupling between them.

5.2 Three-Layer Architecture

Thesystemisorganized into three architectural layers:

Layer 1: Presentation Layer (app.py)

The Presentation Layer is implemented in app.py and is responsible for all user interface concerns. This layer handles:

- Rendering the Streamlit chat interface with custom CSS styling
- Managing the sidebar with brand identity, model selector and student profile
- Displaying conversation history from session state
- Processing user input from the chat input widget and quick-access chips
- Routing responses from the business logic layer to the UI
- Displaying loading spinners and response attribution captions

Layer 2: Business Logic Layer (chatbot.py, llm.py)

The Business Logic Layer is divided across two modules: chatbot.py implements the rule-based intent classification and response generation engine, while llm.py manages the integration with the Groq LLM API. This layer handles:

- Intent matching through the match_intent() function using keyword lookup
- Response formatting through specialized handler functions for each intent
- Building the student context string for LLM system prompts
- Constructing multi-turn conversation history for the LLM
- Making authenticated API calls to the Groq endpoint

- Error handling for LLM API failures and missing configuration

Layer 3: Data Layer (embedded in chatbot.py)

The Data Layer contains the student profile and academic records, currently implemented as Python data structures (dictionaries, lists and tuples) embedded in chatbot.py. This layer provides:

- Student profile: name, roll number, semester and program
- Course enrollment records with course codes, titles and credit values
- Grade records per subject
- Attendance records with sessions attended and total sessions
- Fee records including total, paid and outstanding amounts
- Assignment records with titles, due dates and submission statuses

5.3 Data Flow Diagram

The following describes the flow of data through the system when a user submits a query:

12. The user types a message in the chat input or clicks a quick-access chip.
13. The Presentation Layer appends the message to session state and triggers a UI rerun.
14. The get_response() function in chatbot.py receives the user input string.
15. The match_intent() function scans the input for keywords and returns an intent classification.
16. If the intent is recognized, the corresponding handler function (handle_courses(), handle_grades(), etc.) retrieves data from the Data Layer and formats it as a Markdown string.
17. If the intent is 'unknown', the Presentation Layer calls ask_llm() in llm.py, passing the user message, conversation history and LLM model selection.
18. llm.py constructs the system prompt with student context, builds the message array and sends a request to the Groq API.
19. The response text is returned to the Presentation Layer, displayed in the chat interface and appended to session state.

5.4 Module Descriptions

The following table summarizes the modules and their responsibilities:

Module	Responsibility
app.py	Streamlit UI, session state management, response routing and display
chatbot.py	Student data, intent matching, rule-based response handlers, CLI mode
llm.py	Groq API client, system prompt construction, LLM query and response handling
.env	Environment variable storage for GROQ_API_KEY
requirements.txt	Python package dependencies
.streamlit/config.toml	Streamlit server configuration

6. SYSTEM REQUIREMENTS

6.1 Functional Requirements

Functional requirementsdefinewhatthe system must do - the specific behaviors and functions it must support.

FR1: Course Information Retrieval

The system shall display a tabular list of all courses in which the student is enrolled for the current semester, including course codes, titles and credit values. The total credit count shall be shown at the bottom of the table.

FR2: Grade Information Retrieval

The system shall display the student's latest grades for each enrolled subject in a formatted table, showing course codes, subject names and grade letters.

FR3: Attendance Information Retrieval

The system shall display attendance data for each subject, showing the number of classes attended, total classes held and attendance percentage. Subjects with attendance below 80% shall be flagged with a warning indicator.

FR4: Fee Status Retrieval

The system shall display the student's fee payment status, including total fees, amount paid, outstanding balance and due date. A reminder notification shall be shown if any balance is outstanding.

FR5: Assignment Information Retrieval

The system shall display all assignments with their subject codes, titles, due dates and submission statuses. Pending assignments shall be highlighted and the count of pending items shown.

FR6: Status Dashboard

The system shall provide a consolidated status dashboard showing the student's name, roll number, program, semester, overall attendance percentage, number of pending assignments and outstanding fee balance.

FR7: Natural Language Query Processing

The system shall accept free-form natural language input and classify it against defined intents using keyword matching. Unrecognized queries shall be forwarded to the LLM for processing.

FR8: LLM Fallback

When the LLM API is configured, the system shall pass unrecognized queries to the Groq API with appropriate student context, conversation history and system instructions, and display the generated response to the user.

FR9: Graceful Degradation

In the absence of a GROQ_API_KEY, the system shall display a clear message informing the user that LLM features are unavailable, while continuing to support all rule-based intents.

FR10: Quick-Access Chips

The interface shall provide quick-access chip buttons above the chat input for each primary intent (Courses, Grades, Attendance, Fees, Assignments, Status), allowing one-click access to common queries.

6.2 User Requirements

User requirements describe the needs and expectations of the end users of the system.

- Students shall be able to query their academic information in natural English without knowledge of specific commands.
- The interface shall provide immediate responses to common queries without perceptible delay.

- The system shall present information in a clear, readable format with appropriate visual hierarchy.
- Students shall be able to ask general study questions and receive academically relevant answers.
- The interface shall maintain conversation context so follow-up questions are handled appropriately.
- The system shall work reliably across standard web browsers without requiring plugin installation.
- Students with basic computer literacy shall be able to use the system without training.

6.3 Environmental Requirements

Environmental requirements specify the conditions under which the system must operate.

- Software Environment: The application must run on Python 3.8 or higher. All dependencies must be installable via pip from PyPI.
- Operating System: The application must be compatible with Windows 10+, macOS 11+ and major Linux distributions including Ubuntu 20.04+.
- Browser Compatibility: The web interface must render correctly in Chrome 90+, Firefox 88+, Safari 14+ and Edge 90+.
- Network: LLM features require an active internet connection with access to api.groq.com. Rule-based features must function without internet access.
- Security: The GROQ_API_KEY must be stored in environment variables and never hardcoded in source files or logged to console.
- Accessibility: The interface must use sufficient color contrast ratios and readable font sizes to support accessibility.

7. DESIGN AND ARCHITECTURE

7.1 Intent Classification Design

Thecore of the chatbot's intelligence lies in its intent classification mechanism. The system employs a deterministic keyword-based approach as the primary classification layer. The match_intent() function in chatbot.py processes the user's input string through a series of ordered checks:

20. Empty check: If the input string is empty or contains only whitespace, the 'empty' intent is returned.
21. Exit check: Keywords including 'exit', 'quit', 'bye' and 'goodbye' trigger the 'exit' intent.
22. Help check: Keywords including 'help', 'menu', 'what can you do' and 'commands' trigger the 'help' intent.
23. Domain-specific checks: Ordered checks for keywords corresponding to courses, grades, attendance, fees, assignments and status.
24. Social checks: Keywords for greetings ('hi', 'hello', 'hey', 'namaste') and acknowledgements ('thank').
25. Unknown fallback: If no keyword matches, the 'unknown' intent is returned and the query is forwarded to the LLM.

This ordered approach ensures that common queries are resolved in microseconds through simple string operations, while the LLM is reserved for genuinely complex or novel queries where its capabilities add value.

7.2 Response Generation Design

Each recognized intent is handledbyadedicated function that retrieves data from the Data Layer and formats it as a Markdown string. This design choice ensures that:

- Each response format can be independently tested and modified.
- Streamlit's built-in Markdown rendering produces well-formatted output without additional parsing.

- Tables, bold text, inline code, bullet lists and blockquotes can all be used to create rich, readable responses.

7.3 LLM Integration Design

The LLM integration in llm.py follows a context-injection pattern. Rather than relying on the model's general training knowledge to answer student-specific queries, the system injects the complete student profile - including all courses, grades, attendance records, fee status and assignments - into the system prompt for every API call.

This design decision has several important implications:

- Factual accuracy: The model cannot confabulate student data because it is explicitly provided in the prompt.
- Personalization: Every response can reference the student's actual name, roll number and academic records.
- Consistency: The model's behavior is constrained by the system prompt, which specifies tone, response length and formatting requirements.

The conversation history is maintained in Streamlit's session state and the last six messages are included in each LLM API call, providing sufficient context for multi-turn conversations without exceeding token limits.

7.4 UI/UX Design

The user interface is designed around a dark-sidebar, light-main-panel layout that is common in modern productivity applications. The design system uses a carefully chosen color palette:

CSS Variable	Hex Value	Usage
--bg	#f6f7fb	Main canvas background
--ink	#0d1430	Dark navy — sidebar, borders
--peach	#ff8c5e	Warm accent — buttons, highlights
--mute	#6b7090	Secondary text, labels
--card	#ffffff	Card and bubble backgrounds
--line	#e6e8f0	Borders and dividers

Chat message bubbles use a side-tab design where the assistant's bubble has a 4px left border in the ink color, and the user's bubble has a 4px right border. This subtle visual differentiation helps users quickly identify the direction of each message in the conversation.

7.5 Database Design

In the current implementation, the data layer uses Python data structures. The logical data model is described below:

Student Profile

A dictionary containing name (string), roll_no (string), semester (integer) and program (string). This represents the student's core identity information.

Courses

A list of tuples, each containing (course_code: str, course_title: str, credits: int). This represents the set of courses in which the student is enrolled for the current semester.

Grades

A list of tuples containing (course_code: str, course_title: str, grade: str). Grades are letter grades on a standard A through F scale with plus/minus modifiers.

Attendance

A list of tuples containing (course_code: str, classes_attended: int, total_classes: int). The attendance percentage is computed at runtime.

Fees

A dictionary containing total (int), paid (int), due (int) and due_date (str in ISO 8601 format). All monetary values are in Indian Rupees.

Assignments

A list of tuples containing (course_code: str, title: str, due_date: str, status: str). Status is either 'Pending' or 'Submitted'.

8. IMPLEMENTATION

8.1 Project Structure

Theprojectisorganizedinto a clean, single-directory structure:

```
edtech-chatbot/ ├── app.py                    # Streamlit UI layer ├──
chatbot.py                # Rule-based engine + CLI ├── llm.py
# Groq LLM wrapper ├── requirements.txt          # Python dependencies
├── .env                      # API key (not committed) └── .streamlit/
└── config.toml           # Streamlit configuration
```

8.2 Implementation of chatbot.py

The chatbot.py module forms therule-based core of the system. It begins by defining the student data as module-level constants, making them accessible to both the response handlers and the LLM context builder in llm.py.

The STUDENT dictionary stores the student's identity information. The COURSES, GRADES and ATTENDANCE variables are defined as lists of tuples, providing a natural row-oriented structure for tabular data. The FEES dictionary holds the financial record, and ASSIGNMENTS stores the task list as a list of four-tuples.

Each intent handler function follows the same pattern: build a list of strings representing rows of a Markdown table, join them with newlines, and return the resulting string. For example, handle_courses() constructs a Markdown table with a header row, one row per course, and a footer row showing total credits. The function computes the total credits at runtime using a generator expression over the COURSES list.

The handle_attendance() function includes conditional logic to identify subjects with below-80% attendance. These subjects are flagged with a warning emoji in the table, and a blockquote reminder is appended to the response if any below-threshold subjects exist. This dynamic behavior provides genuinely useful, personalized feedback rather than static data display.

The match_intent() function implements the intent classification logic. It normalizes the input to lowercase, strips leading and trailing whitespace, and then applies a sequence of keyword membership tests. The any() built-in function is used with a list comprehension to efficiently test multiple keywords against the input string. The function returns a string intent label that maps to a handler in the HANDLERS dictionary.

The get_response() function orchestrates the full response pipeline. It calls match_intent() to classify the input, then dispatches to the appropriate handler or returns a special signal for LLM handling. For social intents (greet, thanks), it generates simple inline responses without consulting the handlers dictionary. For the 'unknown' intent, it returns (intent, None), signaling to the Presentation Layer that LLM processing is required.

8.3 Implementation of llm.py

The llm.py module encapsulates all interaction with the Groq LLM API. The _build_student_context() function constructs a structured plain-text summary of all student data, formatted for easy parsing by the language model. This function is called once at module import time to build the SYSTEM_PROMPT constant, avoiding repeated string construction at runtime.

The SYSTEM_PROMPT constant is a multi-part f-string that defines the model's role, behavioral guidelines and student data. It instructs the model to handle two categories of queries: student-data questions (grounded in the provided records) and study questions (using the model's general knowledge with a tutoring style). Style guidelines specify Markdown formatting, a 250-word response limit and first-name addressing.

The is_configured() function provides a simple boolean check for API key availability, enabling the Presentation Layer to update the status pill in the toolbar. The ask_llm() function implements the full API call flow: instantiating the Groq client, constructing the message array with system prompt, conversation history and current user message, invoking the completions API and returning the response text. A try-except block catches all API exceptions and returns a formatted error message rather than propagating the exception to the UI.

8.4 Implementation of app.py

The app.pymodule isthe largest in the project, primarily due to the extensive custom CSS that defines the application's visual identity. The CSS is injected as an HTML style block via st.markdown() at the top of the script, ensuring it is applied before any other UI elements are rendered.

The _init_state() function initializes Streamlit session state on first run, creating the messages list with the chatbot's welcome message and setting the default model. The model selection in the sidebar uses a dictionary (MODEL_LABELS) to map internal model identifier strings to human-readable display labels, keeping the selectbox options clean and presentable.

The sidebar profile card dynamically computes overall attendance percentage and pending assignment count from the Data Layer at render time. These computed values are embedded in an HTML template string and rendered via st.markdown(). This approach bypasses Streamlit's default widget styling, allowing for pixel-precise control over the card's appearance.

The chat message rendering loop iterates over st.session_state.messages and renders each message using st.chat_message(). The message's 'source' field - which contains either 'Rule-based' or 'AI · Llama 3.3' - is rendered as a caption below the assistant's response, providing transparency about how each response was generated.

The quick-access chip implementation uses a horizontal layout of st.button() widgets. When a chip is clicked, its query keyword is stored in st.session_state._pending_input and st.rerun() is called. On the next render cycle, the pending input is retrieved, used as the prompt and cleared from session state using pop().

8.5 Setup and Deployment

Deployingthe EdTech Chatbotrequires the following steps:

26. Clone or extract the project directory to the local machine.
27. Install Python dependencies using: pip install -r requirements.txt
28. Create the .env file by copying the .env.example template.

29. Obtain a free Groq API key from console.groq.com and add it to the .env file as GROQ_API_KEY=gsk_your_key_here.
30. Start the application using: streamlit run app.py
31. Access the application in a web browser at http://localhost:8501.

The application is operational without a Groq API key. In this configuration, rule-based responses for all eight defined intents are fully functional. Only free-form natural language queries that do not match any keyword trigger the LLM fallback, and users are shown a clear message explaining how to enable LLM features.

8.6 Key Code Snippets

Intent Matching Logic

```
def match_intent(user_input):
    text = user_input.lower().strip()
    if not text:
        return "empty"
    if any(k in text for k in ["exit", "quit", "bye"]):
        return "exit"
    if any(k in text for k in ["grade", "marks", "result"]):
        return "grades"
    if any(k in text for k in ["attendance", "present"]):
        return "attendance"
    # ... additional checks ...
    return "unknown"
```

LLM System Prompt Construction

```
SYSTEM_PROMPT = f"""You are an EdTech assistant chatbot for a Computer Science student. Handle two kinds of questions: 1. STUDENT-DATA QUESTIONS- base on data below. 2. STUDY QUESTIONS - answer like a helpful tutor. {_build_student_context()} """
```

9. TESTING

9.1 Testing Strategy

Thetesting strategy forthe EdTech Chatbot encompasses four levels: functional testing of individual intent handlers, integration testing of the intent-to-response pipeline, user acceptance testing of the complete web interface, and performance testing of response times. Each level targets a distinct aspect of system behavior.

9.2 Functional Testing

Functional testsverifythateach intent handler produces the correct output for a given input. The following test cases were executed:

Test ID	Input Query	Expected Intent	Result
TC-01	show my courses	courses	PASS
TC-02	what are my grades	grades	PASS
TC-03	check attendance	attendance	PASS
TC-04	fee balance	fees	PASS
TC-05	pending assignments	assignments	PASS
TC-06	show my status	status	PASS
TC-07	hello	greet	PASS
TC-08	thank you	thanks	PASS
TC-09	bye	exit	PASS
TC-10	(empty string)	empty	PASS
TC-11	explain recursion	unknown (LLM)	PASS
TC-12	am I failing any subject	unknown (LLM)	PASS

9.3 Boundary and Edge Case Testing

Boundaryandedgecasetestsverifysystem behavior under non-standard input conditions:

Test ID	Input Query	Expected Behavior	Result
TC-13	COURSES (uppercase)	Recognizes as courses intent	PASS
TC-14	grades (leading spaces)	Recognizes as grades intent	PASS
TC-15	What are my courses and grades?	Recognizes first match (courses)	PASS
TC-16	xyzabc123 (gibberish)	Returns unknown, forwards to LLM	PASS
TC-17	No API key configured	Shows LLM unavailable message	PASS
TC-18	Network disconnected	API error handled gracefully	PASS

9.4 Customer Testing / User Acceptance Testing

User acceptancetesting was conducted with a group of five BCA students from the Department of Computer Science. Testers were asked to interact with the chatbot freely for fifteen minutes and then complete a usability questionnaire.

The questionnaire used a 5-point Likert scale for the following dimensions:

- Ease of Use: How easy was it to understand how to interact with the chatbot?
- Response Accuracy: Were the responses accurate and relevant to your queries?
- Visual Design: How satisfied are you with the appearance of the interface?
- Response Speed: How satisfied are you with the speed of responses?
- Overall Satisfaction: How satisfied are you with the chatbot overall?

Dimension	Mean Score (out of 5)	Min Score	Max Score
Ease of Use	4.	4	5
Response Accuracy	6	4	5
Visual Design	4.	4	5
Response Speed	4	3	5
Overall Satisfaction	4.	4	5

8

4.

2

4.

5

Qualitative feedback from testers highlighted the visual design, quick-access chips and personalized greeting as particularly appreciated features. Areas for improvement noted by testers included adding voice input, supporting queries across multiple students and integrating with the institution's actual database.

9.5 Performance Testing

Performance testing measured response times under typical usage conditions. Tests were conducted on a standard laptop with a stable Wi-Fi connection.

Query Type	Average Response Time	Maximum Observed
Rule-based intent	< 50 ms	< 100 ms
Chip-triggered query	< 100 ms (with rerun)	< 200 ms
LLM API call (Groq)	1.2 seconds avg	2.8 seconds

Rule-based responses are effectively instantaneous from the user's perspective, limited only by Streamlit's WebSocket round-trip time. LLM response times are primarily determined by the Groq API's inference speed, which was consistently fast due to Groq's optimized hardware.

10. GRAPHICAL USER INTERFACE (GUI) LAYOUT

10.1 Layout Overview

The EdTech Chatbot interface is organized into three primary regions: a fixed-width dark sidebar on the left, a sticky toolbar at the top of the main content area, and a scrollable chat area with a fixed input bar at the bottom.

10.2 Sidebar Design

Thesidebaroccupies 248 pixels of width and features a dark navy background (#0d1430) that provides strong visual contrast with the light main canvas. A subtle grid pattern overlay (created with CSS background-image gradients) adds texture without visual noise.

The sidebar contains the following elements from top to bottom:

- Brand Mark: A 30x30 pixel rounded square displaying 'ed' in bold, accompanied by the wordmark 'EdTech.' with the period in the peach accent color. A 'Academic assistant' tagline appears below the wordmark in muted white.
- New Conversation Button: A full-width button in the peach accent color (#ff8c5e) that clears the conversation history and resets the chatbot to its initial state.
- Divider: A 1px semi-transparent horizontal rule separating the action area from the configuration area.
- AI Model Section: A dropdown selector allowing the user to choose between Llama 3.3 70B (default) and Llama 3.1 8B, with human-readable labels.
- Student Profile Card: A rounded card with a subtle border displaying the student's initial avatar, name, roll number, semester, overall attendance bar and pending assignment count.

10.3 Toolbar Design

Thetoolbar ispositioned at the top of the main content area with a sticky CSS position, ensuring it remains visible as the user scrolls through the conversation. The toolbar uses a backdrop-filter blur effect over a semi-transparent background to create a frosted-glass appearance.

The left side of the toolbar contains the page title ('Your study companion.') with an 'Academic assistant' eyebrow label in small uppercase. The right side shows a status pill that indicates whether the LLM is configured and ready (green dot) or running in rule- based mode (amber dot).

10.4 Chat Area Design

Thechatareais theprimaryworkspace of the application. Messages are rendered using Streamlit's native chat message components, styled with custom CSS to override default appearances.

Assistant messages appear on the left with a 4px navy left border. User messages appear on the right with right alignment and a 4px navy right border. Both message types use a white card background with a subtle shadow, creating depth against the light canvas.

Below each assistant message, a caption shows the response source: 'Rule-based' for intent-matched responses and 'AI - Llama 3.3' for LLM-generated responses. This attribution provides transparency and educational value, helping users understand the system's behavior.

10.5 Quick-Access Chips

Sixpill-shaped buttons appear pinned just above the chat input bar. Each chip corresponds to a primary intent: Courses, Grades, Attendance, Fees, Assignments and Status. The chips have a white background with a navy border in their default state, transitioning to a navy background with white text on hover, accompanied by a subtle upward translation for a lift effect.

10.6 Chat Input Design

Thechat input is a roundedtext field at the bottom of the main area. It has a white background with a light border that changes to the navy ink color when focused, providing clear visual feedback. The submit button uses a navy background with white arrow icon. The input remains fixed at the bottom regardless of conversation length, ensuring it is always immediately accessible.

11. EVALUATION

11.1 Evaluation Framework

The evaluationofthe EdTechChatbot examines the system across four dimensions: technical correctness, user experience quality, architectural soundness and alignment with project objectives. This multi-dimensional assessment provides a comprehensive view of the project's success and limitations.

11.2 Technical Evaluation

Fromatechnical perspective, thesystem achieves its core objectives with high fidelity:

- Intent Classification Accuracy: The rule-based classifier correctly identifies all eight defined intents with 100% accuracy across all test cases, including case-insensitive matching and queries with leading or trailing whitespace.
- Data Integrity: All responses based on the Data Layer accurately reflect the defined student records. No data integrity issues were identified during testing.
- LLM Integration: The Groq API integration functions correctly, with appropriate error handling for missing API keys and network failures. The student context injection reliably grounds LLM responses in the actual student data.
- Session Persistence: Conversation history is correctly maintained across the session using Streamlit session state, enabling coherent multi-turn conversations.
- Graceful Degradation: The system correctly identifies missing API configuration and continues to support all rule-based intents, meeting the graceful degradation requirement.

11.3 User Experience Evaluation

Theuseracceptance testing resultsindicate high satisfaction across all evaluated dimensions, with an overall mean score of 4.5 out of 5. The visual design received the highest rating (4.8), reflecting the quality of the custom CSS implementation. Response accuracy rated 4.4, with testers noting that rule-based responses were precise and LLM responses were generally relevant and helpful.

Areas where the user experience falls short of ideal include: the lack of voice input, the inability to handle queries about multiple students, and the absence of push notifications for approaching deadlines or assignment due dates.

11.4 Architectural Evaluation

The three-module architecture (app.py, chatbot.py, llm.py) demonstrates good separation of concerns. The business logic is fully decoupled from the presentation layer: chatbot.py functions correctly as a standalone CLI application without importing any Streamlit modules. The LLM wrapper is also independently testable without a running Streamlit instance.

The current static data model, while appropriate for a prototype, represents the most significant architectural limitation. Moving to a database-backed data layer would require modifications primarily to chatbot.py, with minimal impact on the other modules - a testament to the architecture's modularity.

11.5 Comparison with Objectives

Objective	Achievement	Status
Hybrid intent classification system	Fully implemented	Achieved
Comprehensive student data model	Fully implemented	Achieved
Streamlit web interface	Fully implemented	Achieved
Groq API integration	Fully implemented	Achieved
Student context injection	Fully implemented	Achieved
Dual deployment (web + CLI)	Fully implemented	Achieved
Modular architecture	Fully implemented	Achieved
Graceful degradation	Fully implemented	Achieved

12. SNAPSHOTS OF THE PROJECT

The following descriptions document the key screens and states of the EdTech Chatbot application. These represent the visual output at various stages of interaction.

12.1 Application Launch Screen

Upon launching the application with streamlit run app.py and opening a browser at http://localhost:8501, the user is greeted with the main chat interface. The dark navy sidebar is visible on the left, displaying the EdTech brand mark, a 'New conversation' button and the student profile card for Fayis. The main area shows the sticky toolbar with the 'Your study companion.' title and the LLM status pill. The chatbot's welcome message is displayed: 'Welcome back, Fayis. I can pull up your courses, grades, attendance, fees and assignments — or help you think through something more open-ended.' The six quick-access chips are pinned above the chat input bar at the bottom.

12.2 Course Information Response

When the user clicks the 'Courses' chip or types 'courses', the chatbot immediately responds with a formatted Markdown table listing all five enrolled courses. The table shows course codes (CS401 through CS405), subject names and credit values, with a total credits row at the bottom. The response is attributed as 'Rule-based' in the caption below the message bubble.

12.3 Attendance Warning Response

When the user queries attendance, the chatbot displays a per-subject attendance table with percentage calculations. CS405 (Software Engineering, 77.8%) is flagged with a warning emoji. A highlighted blockquote at the bottom reminds the student that CS405 is below the 80% threshold and urges regular attendance.

12.4 LLM-Powered Response

When the user asksafree-formquestion such as 'Which subject am I weakest in and how should I prepare?', the chatbot displays a loading spinner while the Groq API processes the request. The response, attributed as 'AI - Llama 3.3', provides a personalized analysis based on the student's grade records. The model correctly identifies CS405 (Software Engineering, grade B) as the weakest subject and provides concrete study recommendations.

12.5 Fee Status Response

The fee statusresponse displaysthe total fees (Rs. 85,000), amount paid (Rs. 70,000) and outstanding balance (Rs. 15,000) in a formatted list. A peach-accented blockquote reminds the student of the due date (May 15, 2026) and urges timely payment.

12.6 Status Dashboard Response

Thestatusdashboard providesa comprehensive single-screen summary in a two-column table format, showing the student's full profile, overall attendance percentage (88.4%), pending assignment count (3) and outstanding fee balance (Rs. 15,000).

12.7 Rule-Based Mode (No API Key)

When accessed without a GROQ_API_KEYin the environment, the status pill in the toolbar shows 'Rule-based mode' with an amber indicator dot instead of the green 'Llama 3.3 - ready' pill. When a free-form question is asked, the chatbot responds with a structured message explaining that the LLM is not configured, providing instructions for obtaining and setting the API key, and noting that all built-in commands continue to function.

13. CONCLUSIONS

13.1 Summary of Achievements

Thisproject successfully designed and developed a Natural Language Interface for EdTech Infrastructure that meets all eight defined objectives. The EdTech Chatbot demonstrates how modern Python tools - Streamlit for rapid web UI development, Groq's fast LLM inference API and a carefully designed intent classification engine - can be combined to produce a polished, functional and intelligent academic assistant.

The hybrid architecture, which combines deterministic rule-based classification with large language model fallback, represents a thoughtful engineering decision that balances speed, accuracy and flexibility. Rule-based responses handle the majority of student queries with near-zero latency and perfect accuracy, while the LLM fallback extends the system's capabilities to handle the long tail of open-ended queries that no finite set of rules can anticipate.

13.2 Key Learnings

Thedevelopmentofthisproject provided valuable learning experiences across multiple domains:

- Natural Language Processing: Practical experience with intent classification, keyword matching and the limitations of rule-based NLP systems. Understanding of when to apply rule-based versus machine learning approaches.
- Large Language Model Integration: Hands-on experience with LLM API integration, prompt engineering, system prompt design and context injection techniques. Understanding of how to ground LLM responses in factual data through prompt construction.
- Web Application Development: Deep experience with Streamlit, including session state management, custom CSS injection, component layout and the reactive programming model that Streamlit employs.
- Software Architecture: Practical application of separation of concerns, modular design and the importance of interface design between components.

Understanding of how good architecture enables independent testing and future extension.

- UX Design: Experience designing and implementing a complete visual design system including color palette, typography, layout and interactive states.

13.3 Challenges Encountered

Several challenges were encountered during development and resolved through research and experimentation:

- Streamlit Session State: Managing conversation history and pending chip inputs across Streamlit's rerun cycle required careful understanding of how session state is preserved and when it is cleared.
- CSS Specificity in Streamlit: Overriding Streamlit's built-in component styles required careful use of CSS specificity and the use of data-testid attribute selectors to target Streamlit-generated DOM elements.
- LLM Prompt Engineering: Crafting a system prompt that reliably produces concise, well-formatted, factually grounded responses required multiple iterations and testing with various query types.
- Chip Positioning: Positioning the chip row at a fixed location above the chat input required understanding the Streamlit DOM structure and using precise CSS positioning relative to known DOM elements.

13.4 Conclusion

The EdTech Chatbot project demonstrates that a capable, visually refined and intelligent academic assistant can be built using open-source tools and free-tier APIs. The project fulfills its stated aims and objectives, produces a fully functional deliverable and provides rich learning experiences across natural language processing, web development, API integration and software architecture. The system is well-positioned for extension into a fully production-ready academic platform.

14. FURTHER DEVELOPMENT OR RESEARCH

14.1 Database Integration

The most impactful near-term enhancement would be replacing the static in-memory data model with a relational database backend. A PostgreSQL or MySQL database could store student records, with SQLAlchemy serving as the ORM layer. This would enable real-time data updates, multi-student support and the ability to reflect changes made by administrators (e.g., newly entered grades or attendance records) immediately in the chatbot.

The modular architecture of the current implementation makes this transition straightforward: the Data Layer in chatbot.py would be replaced with database query functions, while the business logic and presentation layers would require minimal modification.

14.2 Authentication and Multi-User Support

Deploying the systemforan entire studentcohort requires user authentication.
Integrating with the institution's existing authentication system (e.g., LDAP or OAuth2 via Google Workspace) would allow students to log in with their institutional credentials and access their own personalized data. Streamlit's session state and the database backend would need to be keyed on authenticated user identifiers.

14.3 Voice Interface

Addingspeech-to-text input and text-to-speech output would significantly expand accessibility, particularly for students with visual impairments or those who prefer voice interaction. The Web Speech API, available in modern browsers, could provide speech-to-text transcription directly in the browser. The transcribed text would be processed through the existing chatbot pipeline without modification.

14.4 Push Notifications

A proactive notificationsystem would enhance the chatbot's utility by alerting students to approaching deadlines without requiring them to initiate a query. This could be implemented using a scheduled job (e.g., via APScheduler or Celery) that checks assignment due dates and fee deadlines daily, sending email or WhatsApp notifications to students whose deadlines are within a defined threshold.

14.5 Multilingual Support

Supporting regional languages - particularly Malayalam, Kannada and Hindi - would make the chatbot accessible to students who are more comfortable in their native language. The Groq LLM already has multilingual capabilities and can respond in the detected language of the user's input. Extending the rule-based classifier to handle keywords in regional languages (e.g., recognizing 'haajri' as a keyword for the attendance intent) would complete the multilingual experience.

14.6 Advanced Analytics

Adding a faculty-facing analytics dashboard would enable instructors and advisors to monitor aggregate attendance trends, identify at-risk students and track assignment completion rates. Natural language query capabilities could be extended to the faculty dashboard, allowing advisors to ask questions like 'How many students in CS401 have attendance below 75%?' and receive immediate, data-driven responses.

14.7 Integration with External EdTech Platforms

Thesystemcould be extended to integrate with widely used EdTech platforms such as Google Classroom, Moodle or Microsoft Teams. Using these platforms' APIs, the chatbot could retrieve real-time assignment data, schedule updates and course announcements, providing a unified conversational interface to information distributed across multiple systems.

14.8 Semantic Search and RAG

Replacing keyword-based intentmatching with semantic search using sentence embeddings would improve the system's ability to handle varied phrasings of the same

intent. Retrieval-Augmented Generation (RAG) techniques could extend the LLM's knowledge base to include the institution's policy documents, academic calendars and course syllabi, enabling the chatbot to answer questions about exam schedules, holiday calendars and syllabus coverage.

14.9 Mobile Application

Developing a dedicated mobile application using React Native or Flutter would provide a native app experience with push notification support, offline mode for cached data viewing and biometric authentication. The chatbot backend could be exposed as a REST API, with the mobile app serving as an alternative client alongside the Streamlit web interface.

15. REFERENCES

32. Streamlit Inc. (2024). Streamlit Documentation. Retrieved from https://docs.streamlit.io
33. Meta AI. (2024). Llama 3.3 Model Card. Meta Research. Retrieved from https://ai.meta.com/llama/
34. Groq Inc. (2024). Groq API Documentation. Retrieved from https://console.groq.com/docs
35. Brown, T. B., et al. (2020). Language Models are Few-Shot Learners. arXiv preprint arXiv:2005.14165.
36. Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is All You Need. Advances in Neural Information Processing Systems, 30.
37. Python Software Foundation. (2024). Python 3.11 Documentation. Retrieved from https://docs.python.org/3.11/
38. Ratnaparkhe, V., & Senapati, R. (2022). Chatbot for Educational Institutes using NLP. International Journal of Engineering Research and Technology, 11(3), 221-226.
39. Hobert, S. (2019). Say Hello to Your New AI Tutor. In Proceedings of the 14th International Conference on Computer Supported Collaborative Work in Design (pp. 1-6). IEEE.
40. Woebot Health. (2023). Conversational AI in Mental Health and Education. Technical Report. Woebot Labs.
41. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of NAACL-HLT 2019.
42. python-dotenv Contributors. (2024). python-dotenv Documentation. Retrieved from https://pypi.org/project/python-dotenv/
43. Xu, A., Liu, Z., Guo, J., Sinha, V., & Akkiraju, R. (2017). A New Chatbot for Customer Service on Social Media. In Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems. ACM.
44. Adamopoulou, E., & Moussiades, L. (2020). Chatbots: History, Technology and Applications. Machine Learning with Applications, 2, 100006.
45. IBM Corporation. (2023). Natural Language Understanding. IBM Cloud Documentation. Retrieved from https://cloud.ibm.com/docs/natural-language-understanding
46. Lucassen, G., & Dalpiaz, F. (2019). A User-Centred Approach to Chatbot Design. Requirements Engineering, 24(3), 421-436.

16. APPENDIX

Appendix A: Complete requirements.txt

```
streamlit>=1.32.0 groq>=0.11.0 python-dotenv>=1.0.0
```

Appendix B: .streamlit/config.toml

```
[server]    headless = true    port = 8501    [theme] base = "light"
```

Appendix C: Student Data Summary

Thefollowingtablesummarizesthestudentprofile data embedded in the system:

Field	Value
Name	Fayis
Reg No.	23BCAICD059
Program	Computer Science and Engineering
Semester	4
Courses	CS401, CS402, CS403, CS404, CS405 (17 total credits) A,
Grades	A-, B+, A, B
Overall Attendance	88.4% (CS405 below 80% threshold)
Total Fees	Rs. 85,000 (Rs. 15,000 outstanding)
Pending Assignments	3 of 4 assignments pending

Appendix D: Intent-Keyword Mapping

Intent	Trigger Keywords
exit	exit, quit, bye, goodbye
help	help, menu, what can you do, commands

Intent	Trigger Keywords
courses	courses, subjects, enrolled, class list, what am i taking
grades	grade, marks, result, gpa, score
attendance	attendance, present, absent
fees	fee, payment, balance
assignments	assignment, homework, task, submission
status	status, dashboard, overview, summary
greet	hi, hello, hey, namaste
thanks	thank
unknown (LLM)	Any input not matching above

Appendix E: Sample LLM Prompt Structure

The following is a representativeexcerptofthesystemprompt structure sent to the Groq API for each LLM query:

```
[System] You are an EdTech assistant chatbot for a Computer Sciencestudent.
Handle two kinds of questions: 1. STUDENT-DATA QUESTIONS (groundedindata
below) 2. STUDY QUESTIONS (tutor-style explanations) Style: Markdown,
concise (< 250 words), use first name. STUDENT PROFILE Name:Fayis,Roll
No: CS2023045 Program: Computer Science and Engineering, Semester:4
COURSES CS401 Data Structures & Algorithms (4 credits) ... GRADES,
ATTENDANCE, FEES, ASSIGNMENTS [full data injected here] [Conversation
History] User: Which subject am I weakest in? [User] Givemea3-day
study plan for my pending tasks.
```

This structured prompt ensures that the model has complete context to generate factually accurate, personally relevant and appropriately formatted responses for every user query.

--- END OF REPORT ---

Yenepoya Institute of Arts,Science,CommerceandManagement | BCA 2026